SpiceMCP

From a circuit-simulation prototype to an engineering design tool

What changed in versions 0.6 to 0.11, shown with simulations, and what it means for someone designing circuits.



Six headline results. Orange: before this work. Blue: now. Every number is from a simulation and is explained on the page for that step.

The design flow, end to end



Blue boxes are new or substantially improved. You give a spec; SpiceMCP picks a known circuit, sizes it, tunes it in the simulator, turns it into orderable parts, checks how many built boards would pass, and draws the schematic.

How to read this report

- One section per improvement. Each starts with the problem in one sentence, then a **before / after** picture, then what it means for your design.
- **Orange** is always the old behaviour, **blue** the new one, a sand-coloured band is the range the spec allows.
- No simulator knowledge is needed. The few terms that matter (standard values, tolerance, yield, margin) are explained on the last page.

STEP 1 OF 6

Easier to drive: fewer, clearer controls

SpiceMCP is operated through an AI assistant, which picks from a list of commands. The old list was long and loosely described, so the assistant guessed. Guessing is where wrong circuits come from.



Left to right: number of commands, size of the command list, longest description, inputs the assistant had to guess the shape of.

What changed

- 26 commands became 11 (13 today, after steps 3 and 5 added two). Each has a short, strict description and typed inputs.
- Every answer ends with **the next step to take**; every error says how to fix it and shows a correct example.
- The simulator's memory is the record: a past design is always restored from what was simulated, never retyped from memory.

What it means for you

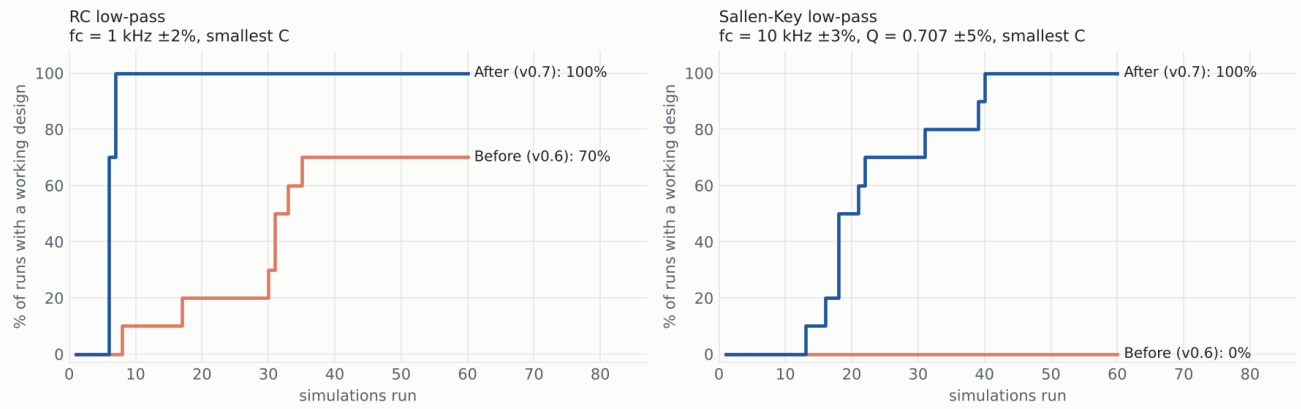
Fewer wrong turns and fewer silently wrong circuits, also with smaller, cheaper AI models.

STEP 2 OF 6

Finds working part values reliably

Part values span decades (100 Ω to 1 MΩ, 1 pF to 1 µF), and several specs pull on the same parts. The old search stepped in straight lines and adjusted one spec at a time, so it often got stuck.

How often the optimizer finds a design that meets every spec (10 runs each)



Share of 10 independent runs that had found a design meeting every spec, against the number of simulations spent.

problem	runs that met every spec	simulations to first working design
RC low-pass, fc = 1 kHz ±2%	7 of 10 → **10 of 10**	31 → **6**
Sallen-Key, fc = 10 kHz ±3% and Q = 0.707 ±5%	0 of 10 → **10 of 10**	never → **20**

What changed

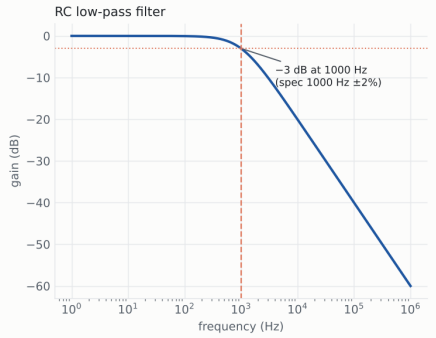
- Steps are taken in ratios (×1.2, ÷1.5), the way an engineer thinks about part values.
- It first samples the whole range evenly, so it does not start in a corner.
- When several specs pull on the same parts (a filter's corner and its Q), it solves for all of them together, from measured sensitivities, instead of one at a time.

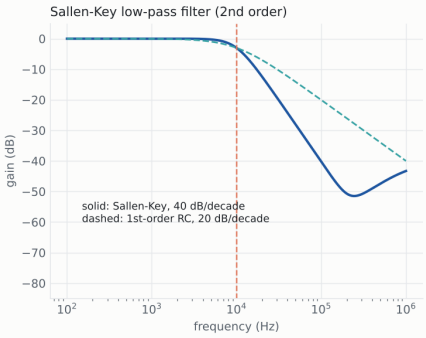
STEP 3 OF 6

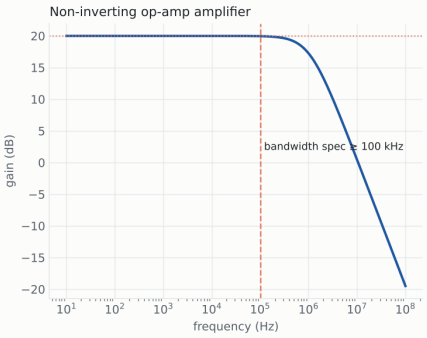
Start from a known circuit, sized by textbook equations

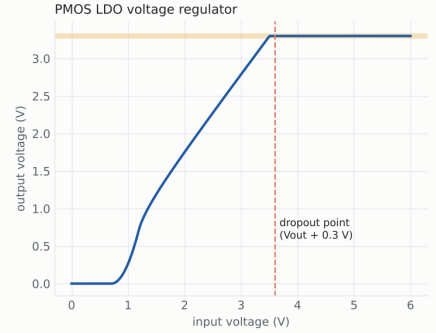
Engineers rarely start from a blank page. Now you can name a circuit and a spec, and get a netlist sized by the standard design equations, already checked in simulation.

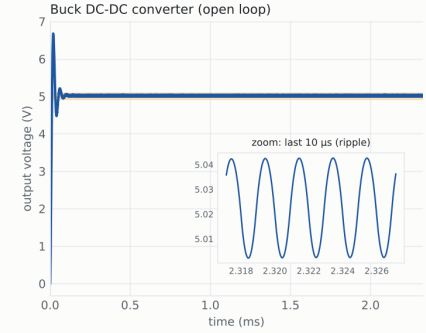
Topology library: five circuits, sized by textbook equations, then simulated









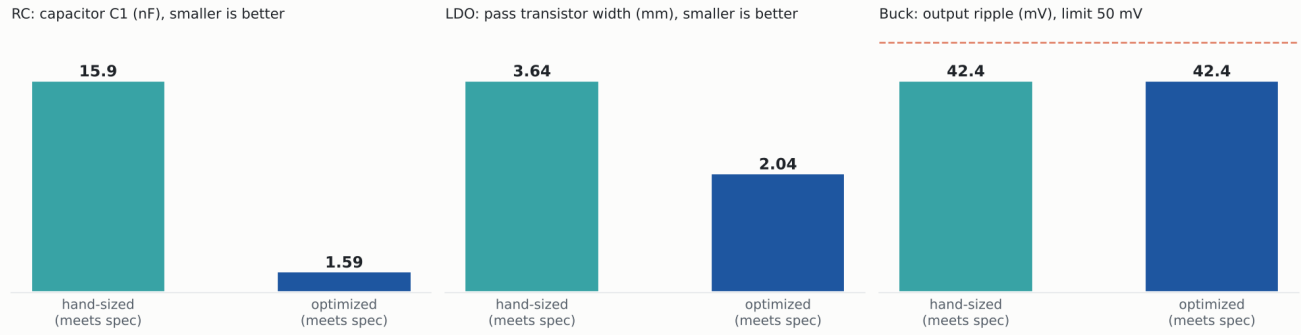


Spec vs simulated, hand-sized values (no optimization)

circuit	metric	spec	simulated
RC low-pass	fc	1000 Hz	1000.0 Hz
Sallen-Key	fc (−3 dB)	10000 Hz	9998 Hz
Sallen-Key	gain at fc/2	≥ −0.5 dB	-0.24 dB
Amplifier	gain	20.0 dB (10×)	20.00 dB
Amplifier	bandwidth	≥ 100 kHz	1060 kHz
LDO	Vout at 5 V in	3.30 V ±1%	3.301 V
LDO	Vout at 3.6 V in	≥ 3.234 V	3.300 V
Buck	Vout	5.0 V ±2%	5.023 V
Buck	ripple	≤ 50 mV	42.4 mV
Buck	efficiency	(reported)	94.6 %

All five ready-made circuits at their hand-calculated values, simulated. The table (bottom right) compares spec and simulation.

The optimizer refines the hand-sized start (40 simulations each)



Tuning then improves the hand-sized start: a 10× smaller capacitor in the RC filter, 14% smaller C1 in the Sallen-Key, a 44% smaller pass transistor in the LDO.

What the five circuits are

circuit	you give	sized by
RC low-pass	corner frequency	fc = 1 / (2πRC)
Sallen-Key low-pass (2nd order)	corner frequency, Q	equal-R design: C1 = 2Q/(ωR), C2 = 1/(2QωR)
Non-inverting amplifier	gain, bandwidth, op-amp GBW	R2 = R1·(G−1); checks GBW ≥ G·bandwidth
PMOS LDO regulator	Vin, Vout, load current, dropout	divider from Vref; pass device from current and dropout
Buck converter	Vin, Vout, Iout, switching frequency, ripple	duty = Vout/Vin; L from ripple current; C from ripple voltage

What it means for you

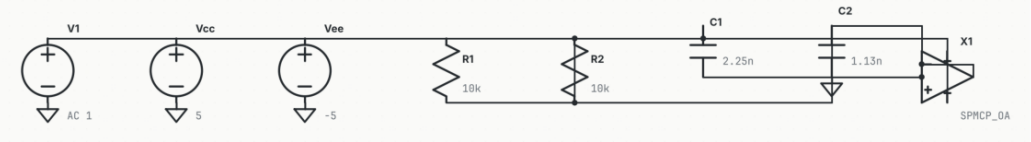
- A working start in one step, with the hand calculation shown line by line, so you can check the reasoning.
- Impossible specs are refused up front with the reason (for example, a gain-bandwidth product the op-amp cannot reach, or a dropout below what the pass device allows).
- Tuning starts close to the answer, so it spends its simulations improving the design rather than searching for one.

STEP 4 OF 6

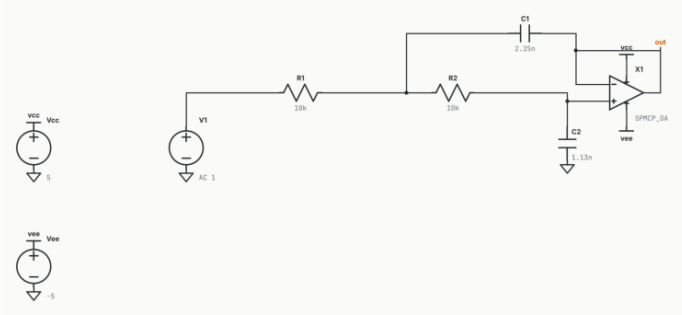
Schematics drawn automatically, and proven correct

A netlist is a list of connections, not a picture. Before, without someone placing every part by hand, parts landed in a row and straight wires joined them. In the simulator, a wire that crosses a pin connects to it, so those drawings were quietly a **different circuit**.

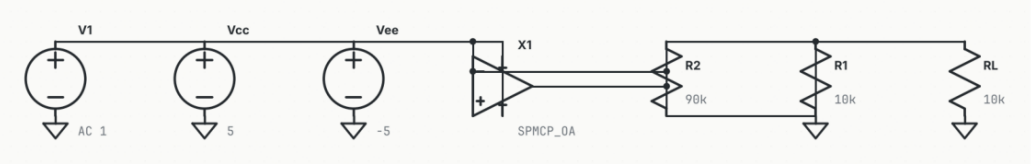
Sallen-Key low-pass filter. BEFORE: parts in a row. wrong circuit: 20 node(s) shorted or broken



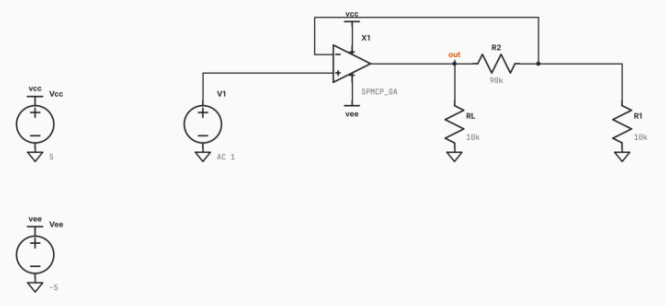
Sallen-Key low-pass filter. AFTER: drawn automatically. same circuit as the netlist



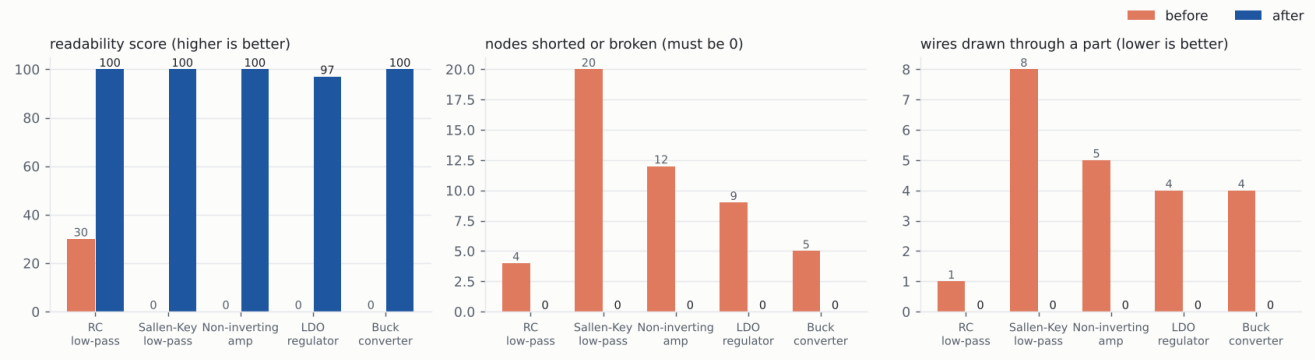
Non-inverting amplifier. BEFORE: parts in a row. wrong circuit: 12 node(s) shorted or broken



Non-inverting amplifier. AFTER: drawn automatically. same circuit as the netlist



Orange title: the old way. Blue title: drawn automatically. Each title says whether the drawing is still the circuit in the netlist. In the old drawings, look for wires running straight across resistor and capacitor pins.



circuit	before: score	before: nodes shorted or broken	after: score	after: nodes shorted or broken
RC low-pass	30	4	**100**	**0**
Sallen-Key low-pass	0	20	**100**	**0**
Non-inverting amp	0	12	**100**	**0**
LDO regulator	0	9	**97**	**0**
Buck converter	0	5	**100**	**0**

Readability score: 100 minus points for each wire crossing, wire through a part, overlapping part, hidden connection and dangling wire.

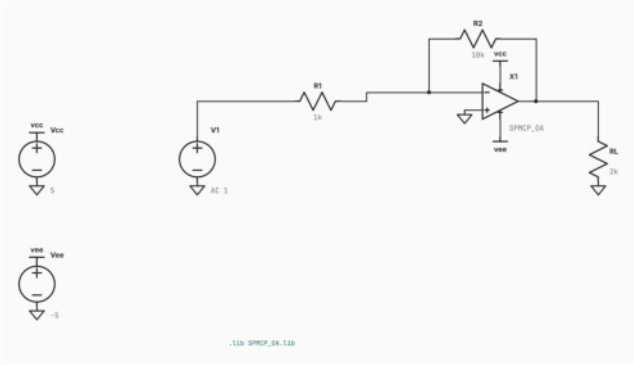
How the drawing is laid out

- Signal flows left to right; an amplifier's inputs are always left of its output.
- Ground at the bottom: every grounded part hangs down and gets its own ground symbol.
- Feedback parts (a feedback resistor, a Sallen-Key's C1) get their own lane above or below the main path. Transistor amplifiers stand upright, drain above source.
- Wires are right-angled, never pass through a part, and never touch another net. Measured nodes (out, fb, ref) are labelled.
- **Every drawing is read back and compared with the netlist, pin by pin.** A drawing that is not the same circuit is refused, never shown.

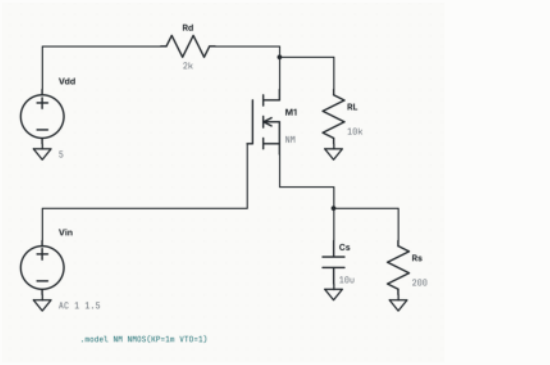
It works on circuits it was never tuned for

The layout rules were written against the five circuits above. These were tried only afterwards (scores 99 to 100, no crossings):

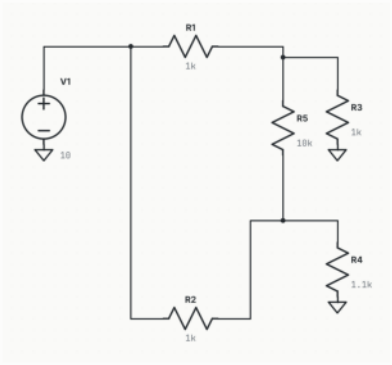
Inverting amplifier · score 100/100 · 0 crossings



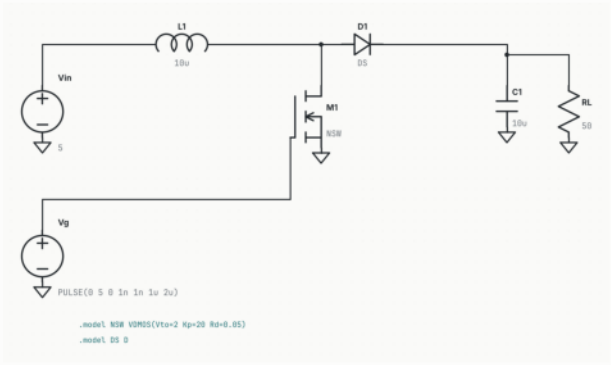
Common-source MOSFET amp · score 99/100 · 0 crossings



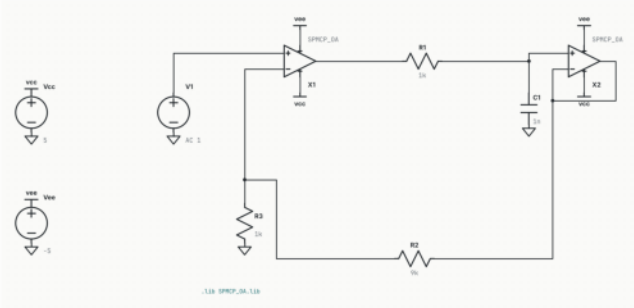
Wheatstone bridge · score 99/100 · 0 crossings



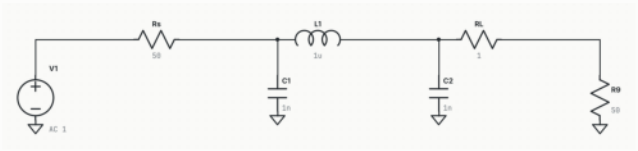
Boost converter · score 100/100 · 0 crossings



Two-stage amp with feedback · score 100/100 · 0 crossings



Pi filter · score 100/100 · 0 crossings



STEP 5 OF 6

From ideal values to a parts list you can order

Tuning produces values like C1 = 1.5915 nF. Nobody sells that. Rounding every part to the nearest standard value moves all of them at once, and on a real board every part is also off by its tolerance. A design that sits on the edge of its spec passes on paper and fails on many boards.

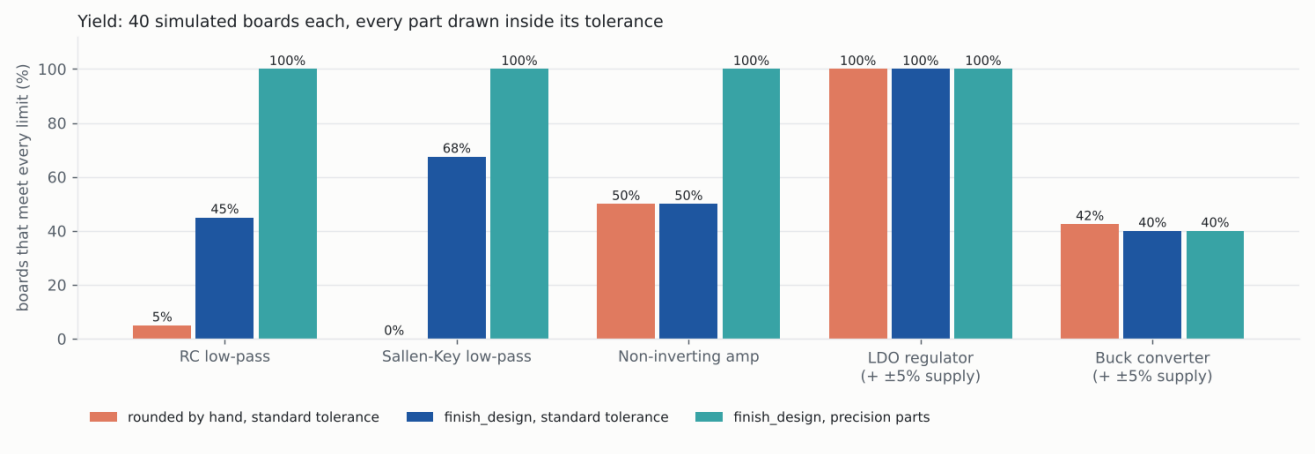
circuit	part	optimiser value (not a real part)	rounded by hand (nearest standard)	SpiceMCP finish_design
RC low-pass	C1	1.59n	1.5n	1.8n (E12, ±5%)
	R1	100k	100k	88.7k (E96, ±1%)
		limits met?	NO	YES
Sallen-Key low-pass	C1	1.95n	1.8n	2.2n (E12, ±5%)
	C2	1.13n	1.2n	1n (E12, ±5%)
	R1	10k	10k	13.3k (E96, ±1%)
	R2	10k	10k	9.31k (E96, ±1%)
		limits met?	NO	YES
Non-inverting amp	R1	10k	10k	10k (E96, ±1%)
	R2	90k	90.9k	90.9k (E96, ±1%)
		limits met?	YES	YES
LDO regulator	R1	175k	174k	174k (E96, ±1%)
		limits met?	YES	YES
Buck converter	C	1.88u	1.8u	2.2u (E12, ±5%)
	L	20.3u	22u	22u (E12, ±20%)
		limits met?	YES	YES

Rounding by hand breaks the two filters: the corner frequency leaves its window. The new finish step finds standard-value sets that meet every limit for all five.

How the finish step works

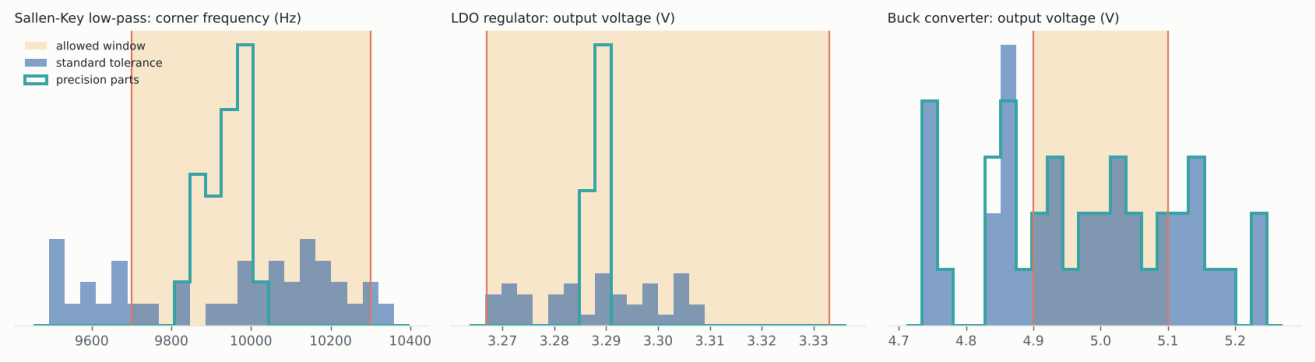
- Pick the coarse-step parts first (capacitors come in E12: 1.0, 1.2, 1.5, 1.8, 2.2 ...), trying the standard value just below and just above.
- Re-centre the fine-step parts (1% resistors in E96) so the design sits in the middle of its limits, the way you would recompute R after choosing C.
- Build 30–40 virtual boards with every part at a random value inside its tolerance, plus the supply within ±5% where it matters, and keep the set that passes most often.

Yield: how many built boards would meet the spec



circuit	rounded by hand	finish step	finish step, precision parts
RC low-pass	5%	**45%**	100%
Sallen-Key low-pass	0%	**68%**	100%
Non-inverting amp	50%	**50%**	100%
LDO regulator (+ ±5% supply)	100%	**100%**	100%
Buck converter (+ ±5% supply)	42%	**40%**	40%

Standard tolerance: 1% resistors, 5% capacitors, 20% inductors. Precision parts: 0.1% resistors, 1% capacitors, 10% inductors. Both columns use the same virtual boards, so the comparison is fair; 40 boards resolve yield to about ±8%.



Where the boards land. Shaded band: allowed window. Filled: standard tolerance. Outline: precision parts.

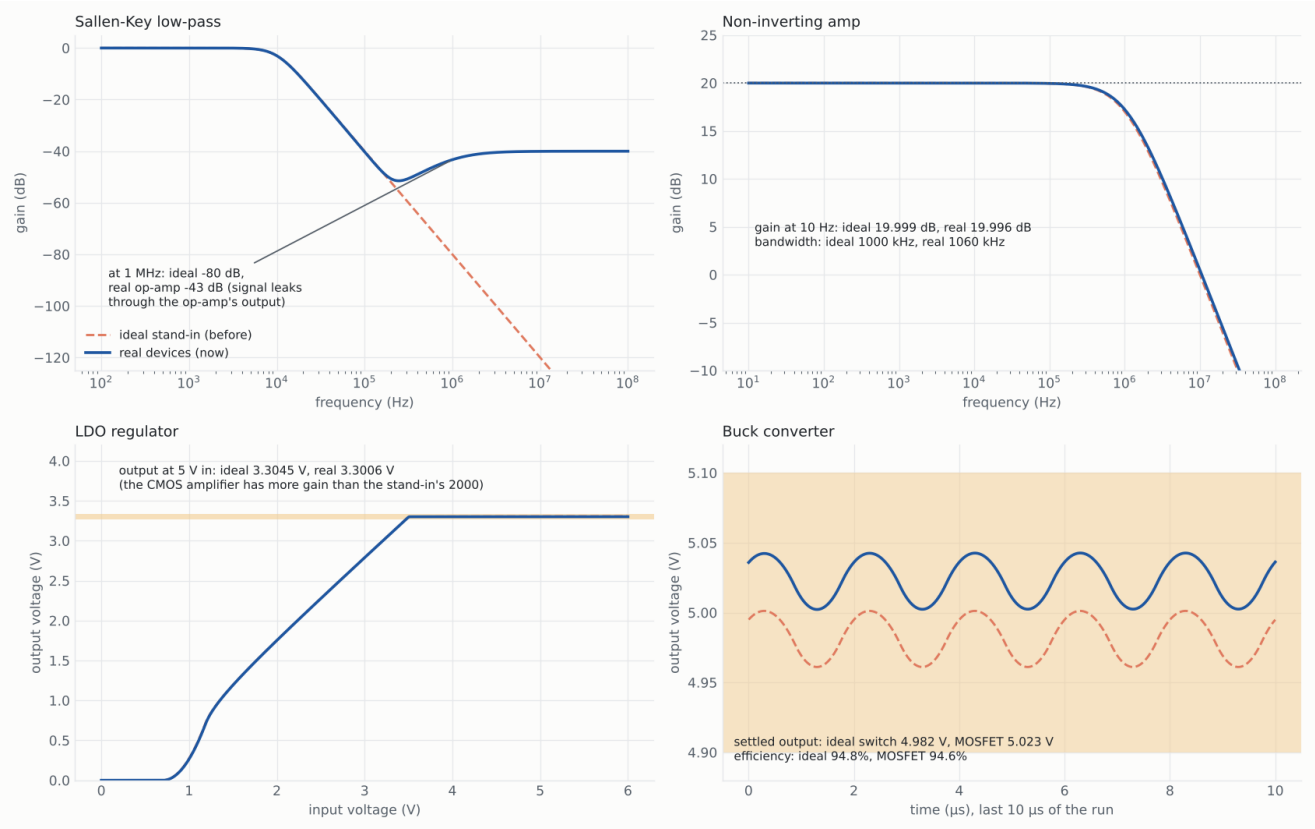
What the yield numbers tell a designer

- **Filters.** A ±2–3% corner frequency cannot be held with 5% capacitors, whatever values you choose. The finish step gets the most out of them (Sallen-Key 0% → 68% of boards), and 1% capacitors take it to 100%.
- **Amplifier.** ±0.1 dB of gain is ±1.2%. Two 1% resistors can be 2% off in ratio. Use 0.1% resistors (100%) or loosen the spec.
- **LDO.** Its feedback loop holds 3.3 V against ±5% supply and 1% resistors: 100%.
- **Buck.** This ready-made buck runs without feedback, so its output follows its input: ±5% in gives ±5% out against a ±2% spec. Better parts do not help (40%). The fix is a control loop, and the yield check shows that before a board is built.

STEP 6 OF 6

Real op-amps and switches, not ideal stand-ins

The amplifier circuits used to model the op-amp as a dependent voltage source: one number for its gain, no supply, no limits. The buck used an ideal switch. Quick for a first look, but it hides what a real part does.



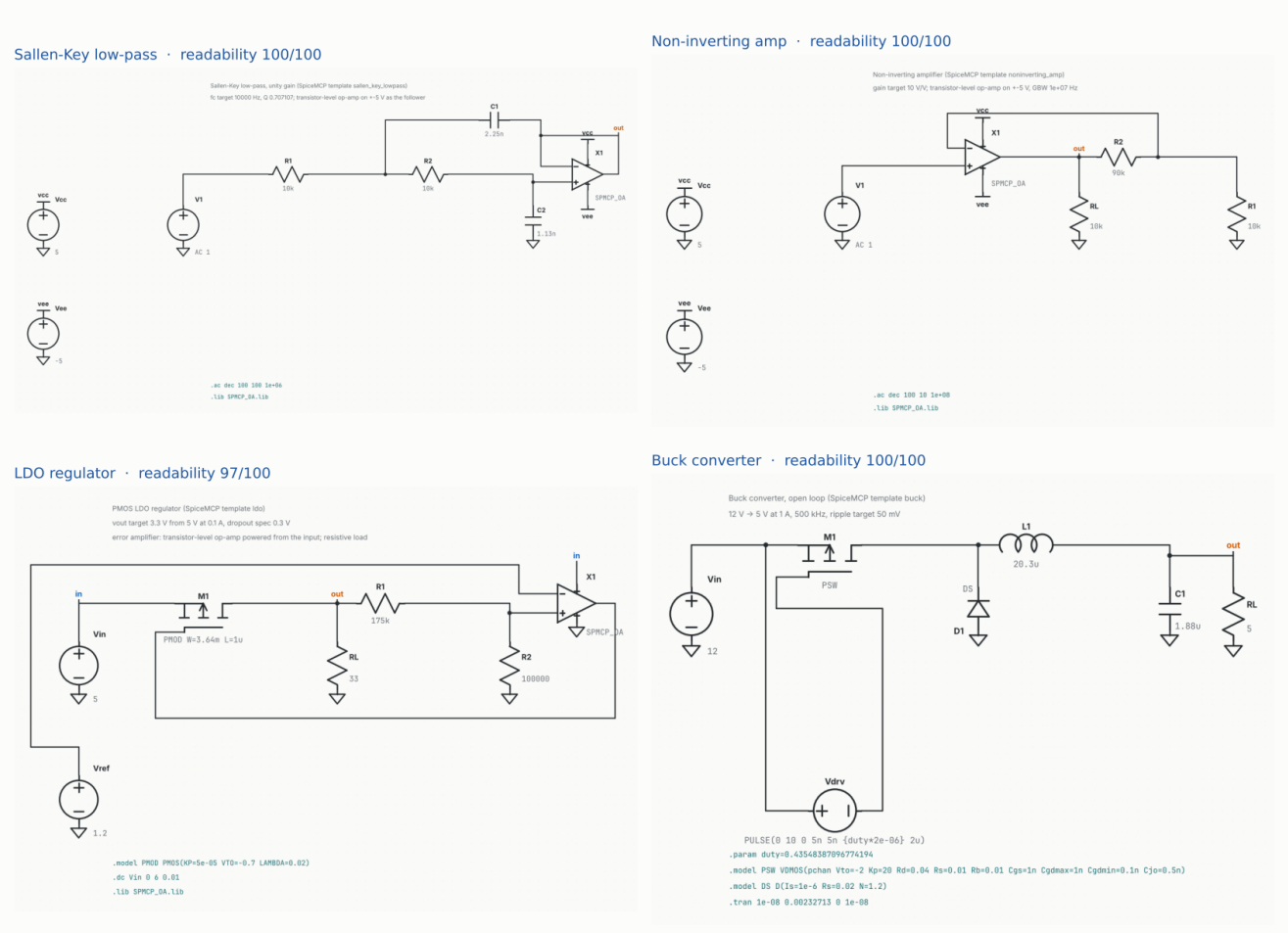
Same circuits, same values. Dashed orange: the ideal stand-in used before. Solid blue: real devices, used now.

What replaced the stand-ins

- **Op-amp** (Sallen-Key, amplifier, LDO error amplifier): a two-stage CMOS op-amp made of eight MOSFETs, running from its own supply pins. Measured: 87 dB open-loop gain, 10.6 MHz gain-bandwidth, 10 V/µs slew, 1.1 mA supply current. It has the standard 5-pin layout, so a vendor model can be swapped in.
- **Switch** (buck): a P-channel power MOSFET (50 mΩ, 1 nF gate) with a gate driver.

What the real parts show that the stand-ins hid

- **Sallen-Key.** At 1 MHz the ideal model claims -80 dB. The real op-amp gives -43 dB, because signal leaks through the feedback capacitor into the op-amp's output. If the filter must block a clock or switching noise, only the real model gives the right answer.
- **Amplifier.** Gain and bandwidth agree within a few percent. The real op-amp adds what the stand-in left out: supply current, output swing and slew rate.
- **LDO.** Both regulate. The real amplifier has more gain, so the output is closer to 3.3 V (3.3006 V vs 3.3045 V).
- **Buck.** The MOSFET's gate charge stretches every on-time slightly: 4.98 V becomes 5.02 V. The sizing equations now keep 20% ripple margin for it.
- All five ready-made circuits still meet their spec at the hand-sized start.



The schematics now show the real parts: the standard op-amp symbol with supply labels, and the power MOSFET. The op-amp's circuit is saved next to the drawing, so it opens and simulates in LTspice as it is.

Where it stands, honestly

- Everything in this report was simulated with ngspice, a second simulator, so the results do not depend on one tool. LTspice remains the main target on Windows and reads the same circuit files.
- The automatic schematic handles flat circuits and 5-pin op-amps. Other sub-blocks (subcircuits) are refused with a clear message rather than drawn wrongly.
- The transistor-level op-amp uses simple MOSFET models, so its phase margin is optimistic. For a stability sign-off, use the vendor model of the part you will buy.
- Yield covers part tolerance and supply. Temperature and silicon process corners need real device models; the design report can run them where models are supplied.
- Test suite: 283 automated checks pass. Another 41 depend on reference files produced by LTspice on Windows, which are not stored in the repository.

Five terms, in plain words

term	meaning
Standard values (E-series)	The values parts are actually sold in. E12 has 12 per decade (1.0, 1.2, 1.5 ... 8.2), E96 has 96 (1.00, 1.02, 1.05 ...).
Tolerance	How far a real part may be from its marked value: a 5% 10 nF capacitor is somewhere between 9.5 and 10.5 nF.
Yield	The share of built boards that meet every spec, with every part somewhere inside its tolerance.
Margin	How far the design sits inside its nearest limit. Zero means on the edge: half the boards would fail.
Netlist	The text list of which part connects to which node: what the simulator reads, and what the schematic must match.